Implementacion de clases en C++
ChessAI

Samuel Thomas Vásquez Gerena

1. Implementación de la Clase `Position`

La clase `Position` gestiona las coordenadas (fila y columna) del tablero. Siguiendo las mejores prácticas de C++, su arquitectura se divide en un archivo de cabecera (`.hpp` o "contrato") y uno de implementación (`.cpp` o "lógica").

Para garantizar tanto la seguridad estructural como el máximo rendimiento, la clase aplica los siguientes principios:

- **Encapsulamiento e Inicialización:** Las variables `row` y `col` son estrictamente privadas. Se inicializan de forma ultra rápida mediante "listas de inicialización" en el constructor y solo pueden leerse desde el exterior usando métodos getters.
- **Inmutabilidad (const):** Todos los métodos de lectura están marcados como `const`, garantizándole al sistema que consultar una posición jamás alterará sus datos internos.
- **Paso por Referencia (`const Position&`):** Para evaluar si dos casillas son iguales, el sistema no hace copias pesadas en la memoria; simplemente lee la dirección original de la otra posición de forma protegida.
- **Sobrecarga del Operador (==):** Se redefinió el símbolo de igualdad para permitir comparaciones matemáticas naturales (ej. `posA == posB`). Internamente, esto se apoya en el puntero explícito `this->` (que significa "mis propias coordenadas"). Esta optimización es vital para que la IA Minimax pueda evaluar millones de posiciones por segundo con un código limpio y eficiente.

2. Implementación de la Clase `Move`

La clase `Move` actúa como un Objeto de Transferencia de Datos (DTO, por sus siglas en inglés) que encapsula toda la información necesaria para ejecutar y revertir un turno dentro del motor de ajedrez. Su diseño evolucionó de ser un simple contenedor de coordenadas a convertirse en una verdadera *Caja Fuerte de Memoria*, indispensable para soportar el funcionamiento del algoritmo *Minimax*.

Para garantizar seguridad de memoria y un alto rendimiento durante la simulación de jugadas, la clase aplica los siguientes principios:

- **Gestión Segura de Memoria (Eliminación de Dangling Pointers):** En lugar de utilizar punteros crudos (`Piece*`) para referenciar piezas capturadas, la clase emplea `std::unique_ptr<Piece>`. Esto garantiza que, cuando el tablero sobrescribe una casilla durante una captura, la pieza enemiga no se destruye definitivamente. En su lugar, la propiedad temporal de la pieza capturada se transfiere al objeto `Move`, preservando su existencia para una posible operación de deshacer.

- **Encapsulamiento de la Operación Inversa:** La clase implementa los métodos `setCapturedPiece(std::unique_ptr<Piece>piece)` y `releaseCapturedPiece()`. Estos métodos permiten que el tablero transfiera la propiedad de la pieza capturada al objeto `Move` al ejecutar un movimiento, y posteriormente recupere dicha pieza cuando se realiza la operación inversa del movimiento.

- **Alta Cohesión del Movimiento:** Cada objeto `Move` es completamente dueño de su propio contexto histórico. Al almacenar internamente la pieza que capturó, evita que otras clases del sistema (como `Game` o `Board`) tengan que mantener estructuras externas para gestionar piezas capturadas. Esta decisión simplifica significativamente la lógica necesaria para soportar la recursividad profunda del motor de Inteligencia Artificial.

- **Soporte para la Simulación del Motor IA:** Gracias a este diseño, cada movimiento contiene toda la información necesaria para ejecutarse y revertirse de forma segura. Esto permite que el algoritmo *Minimax* explore el árbol de decisiones moviéndose hacia adelante y hacia atrás en el estado del tablero sin realizar copias costosas ni recurrir a asignaciones dinámicas innecesarias.

3. Implementación de la Clase `Board`

La clase `Board` representa la capa física del motor de ajedrez. Su responsabilidad exclusiva es actuar como el *Trabajador de la Memoria*, administrando la matriz bidimensional `grid[8][8]` de punteros inteligentes. El tablero es completamente ciego a las reglas del ajedrez: no conoce turnos, jaques ni colores; simplemente confía en que todas las jugadas que recibe ya han sido validadas por las capas superiores del sistema.

Para permitir que la Inteligencia Artificial explore árboles de decisión profundos sin colapsar la memoria RAM, se implementa el patrón arquitectónico de **Inverse Operation (Deshacer Físico)** en lugar de clonar el estado completo del tablero mediante *snapshots*.

- **Paso por Referencia (`Move& m`):** Los métodos físicos del tablero reciben un objeto `Move` por referencia no constante en lugar de coordenadas sueltas. Esto permite que la clase `Board` modifique el objeto durante la ejecución del movimiento, inyectando información crítica como la pieza capturada sin generar copias costosas de datos.

- **Ejecución Física del Movimiento (`movePiece`):** El método `movePiece(Move& m)` ejecuta el movimiento físico utilizando la semántica de transferencia de propiedad mediante `std::move`. Si detecta un puntero no nulo en la casilla de destino (indicando una captura), primero transfiere ese `unique_ptr` al objeto `Move` mediante `m.setCapturedPiece()`, salvaguardando la memoria. Posteriormente traslada la pieza atacante a la nueva posición.

- **Operación Inversa (`undoPiece`):** El método `undoPiece(Move& m)` ejecuta la operación física exactamente en sentido inverso. La pieza atacante se devuelve a su posición original y, si el objeto `Move` indica que hubo una captura mediante `m.isCapture()`, el tablero ordena al objeto liberar el `unique_ptr` de la pieza capturada de regreso a la matriz.

- **Exploración Eficiente del Árbol de Juego:** Gracias a esta mecánica de movimiento y deshacer movimiento, el algoritmo *Minimax* puede explorar estados futuros y retroceder en el tiempo con un coste extremadamente bajo. El sistema evita completamente la clonación del tablero y elimina llamadas innecesarias a memoria dinámica como `new` o `delete` durante la búsqueda.

4. Implementación de la Clase `Piece`

La clase `Piece` funciona como la clase base abstracta para todas las piezas del motor de ajedrez. Siguiendo los principios de la Programación Orientada a Objetos en C++, define el "contrato" central que cada pieza específica (Peón, Caballo, etc.) debe cumplir. Debido a su naturaleza abstracta, su archivo de implementación (`.cpp`) es intencionalmente minimalista.

Para garantizar un diseño polimórfico seguro, escalable y libre de errores de memoria, la clase aplica los siguientes principios técnicos:

- **Abstracción Pura (= 0):** El método `getPossibleMoves` está declarado como virtual puro. Esto hace imposible instanciar una "Pieza" genérica por error y obliga contractualmente a que cada subclase programe sus propias reglas matemáticas de movimiento.
- **Seguridad de Tipos (`enum class`):** El bando de la pieza se gestiona mediante un enumerador fuertemente tipado (`Color::WHITE`, `Color::BLACK`). Esto reemplaza el uso de números enteros tradicionales (0 o 1), haciendo el código más legible y eliminando por completo los bugs por conversiones matemáticas accidentales.
- **Herencia Controlada (`protected`):** A diferencia de las clases puramente encapsuladas, el atributo de color es protegido (`protected`). Esto lo mantiene completamente oculto de sistemas externos (como la interfaz gráfica o el `Board`), pero permite que las clases hijas accedan a él de forma directa.
- **Gestión de Memoria Segura (`virtual P̃iece`):** Se implementa un destructor virtual explícito. Esta es una regla inquebrantable en C++ al usar herencia: garantiza que, cuando el motor elimine una pieza del tablero usando polimorfismo, el sistema destruya correctamente la subclase específica, evitando cualquier fuga de memoria (Memory Leak).
- **Valoración Relativa (`getValue`):** Mediante el método virtual puro `virtual int getValue() const = 0;`, se obliga a cada pieza derivada a definir su peso numérico estándar (ej. Peón = 1, Torre = 5, Reina = 9). Esta abstracción establece la base matemática para calcular la ventaja material del tablero y evaluar posiciones en futuros algoritmos de inteligencia artificial.

4.1. Implementación de la Subclase `Pawn` (Peón)

La clase `Pawn` es la única pieza del motor de ajedrez con un comportamiento de movimiento asimétrico y no reversible. A diferencia del resto del ejército, su capacidad de desplazamiento frontal es estrictamente distinta a su capacidad de captura diagonal.

Para gestionar esta complejidad geométrica con un rendimiento óptimo, la clase aplica los siguientes principios de diseño:

- **Asimetría Direccional Dinámica (Operadores Ternarios):** En lugar de duplicar la lógica con bloques `if (blancas) {...} else {...}`, el algoritmo calcula dinámicamente el vector de avance (`direction: -1 o +1`) y la fila de origen (`startRow: 6 o 1`) en una sola línea. Esto unifica la matemática para ambos colores.
- **Validación de Bloqueo Anidada:** El movimiento de dos casillas iniciales está estructuralmente subordinado a la validación de la primera casilla. Si la primera casilla está bloqueada, el motor no pierde ciclos de CPU calculando el salto doble, garantizando la física real del juego (el peón no salta).
- **Segregación de Responsabilidades Físicas:** El código divide explícitamente el "Movimiento Pacífico" (avance recto condicionado a casillas vacías) de las "Capturas" (avance diagonal condicionado a la existencia de enemigos).
- **Escalabilidad Prevista:** La estructura lineal de las evaluaciones deja el terreno preparado para inyectar en el futuro las reglas especiales de Captura al Paso (En Passant) y Coronación (Promotion) al final de la rutina.

4.2. Implementación de la Clase `Rook` (Torre)

La clase `Rook` introduce la mecánica de las piezas "deslizantes" (Sliding Pieces). A diferencia del Peón, la Torre no tiene un límite fijo de casillas por turno; su movimiento se proyecta como un rayo continuo hasta encontrar una interrupción física o los límites del tablero.

Para resolver esta mecánica de forma eficiente y matemáticamente limpia, la clase aplica los siguientes principios de diseño:

- **Vectores de Dirección Ortogonal:** Utiliza una matriz de constantes (`directions[4][2]`) para definir matemáticamente los ejes cardinales (Arriba, Abajo, Izquierda, Derecha). Esto elimina la necesidad de escribir cuatro bloques de código redundantes, unificando la lógica en un solo bucle iterador.
- **Proyección de Rayos (Bucle `while`):** Implementa un bucle indefinido `while (true)` dentro de cada vector direccional. Esto permite que la pieza "patine" libremente por las casillas vacías, registrando cada paso secuencialmente (`currentRow += dir[0]`).
- **Resolución de Colisiones Perimetrales:** El primer filtro de seguridad verifica los límites de la matriz mediante `!board.isValidPosition`. Si el rayo sale del tablero, el bucle se rompe (`break`), evitando errores críticos de memoria (Segmentation Fault).

- **Bloqueo Físico y Captura Terminal:** Al detectar una casilla ocupada, el algoritmo evalúa la propiedad de la pieza. Si es enemiga, registra el movimiento como una captura válida. Inmediatamente después —sin importar el bando de la pieza detectada— ejecuta un `break` incondicional, garantizando la regla física fundamental de la Torre: no puede saltar sobre otras piezas.

4.3. Implementación Clase `Bishop` (Alfil)

La clase `Bishop` comparte la misma arquitectura de proyección continua (Sliding Piece) que la Torre, pero altera fundamentalmente su plano de interacción física. En lugar de moverse por los ejes ortogonales, el Alfil proyecta su influencia exclusivamente a través de las diagonales de la matriz.

Para implementar esta variación manteniendo la eficiencia del motor, la clase aplica los siguientes principios:

- **Vectores de Proyección Diagonal:** Modifica la matriz de constantes direccionales (`directions[4][2]`) utilizando únicamente combinaciones simétricas de `1` y `-1`. Esto garantiza que por cada paso que la pieza da en el eje X (columnas), avanza matemáticamente un paso idéntico en el eje Y (filas), generando un movimiento diagonal perfecto.

- **Consistencia del Algoritmo de Raycasting:** Reutiliza intacto el motor de cálculo mediante el bucle `while (true)`. Al igual que la Torre, el Alfil evalúa secuencialmente las casillas en su trayectoria hasta encontrar una condición de salida (`break`), ya sea por colisión con el límite de la matriz (`!isValidPosition`) o por intersección con masa física (piezas aliadas o enemigas).

- **Validación de Captura Diagonal:** Aplica la misma regla de resolución de colisiones: si el rayo detecta una pieza, evalúa su color. Si es enemiga, registra el movimiento como una captura legítima antes de detener incondicionalmente la proyección escalar.

4.4. Implementación de la Clase `Queen` (Reina / Dama)

La clase `Queen` representa el ápice del sistema de "piezas deslizantes" (Sliding Pieces). Arquitectónicamente, no introduce un paradigma de movimiento nuevo, sino que fusiona las capacidades de proyección ortogonal de la Torre y la proyección diagonal del Alfil en un solo algoritmo omnidireccional.

Para lograr esta versatilidad manteniendo el coste computacional al mínimo, la clase aplica los siguientes principios de diseño:

- **Fusión Vectorial (Omnidireccionalidad):** Consolida los 4 vectores direccionales de la Torre y los 4 del Alfil en una única matriz unificada de 8 direcciones (`directions[8][2]`). Esto permite que un solo iterador recorra los 360 grados de movimiento posible de la pieza.

- **Reutilización del Motor de Raycasting:** Mantiene exactamente la misma estructura de bucle `while (true)` utilizada en sus contrapartes deslizantes. La Reina proyecta su influencia casilla por casilla en cada uno de los 8 vectores hasta encontrar una condición de parada.

- **Resolución de Colisiones Uniforme:** Aplica la misma validación de límites de matriz (`!isValidPosition`) e intersección de masa física. Al igual que la Torre y el Alfil, la Reina no puede saltar obstáculos; si detecta una pieza enemiga, registra la captura e interrumpe inmediatamente la proyección en ese vector (`break`).

4.5. Implementación de la Clase `King` (Rey)

La clase `King` marca la transición hacia las piezas de movimiento discreto o paso fijo (Step Pieces). Aunque estratégicamente es la pieza más importante (cuyo asedio define la condición de victoria), físicamente posee la matriz de movilidad más fundamental del tablero: un solo paso en cualquier dirección.

Para implementar esta mecánica descartando el motor algorítmico de las piezas deslizantes, la clase aplica los siguientes principios de diseño:

- **Omnidireccionalidad Limitada:** Reutiliza la matriz direccional completa de 8 vectores (`directions[8][2]`) empleada por la Reina, pero omite el bucle de proyección (`while`). Esto restringe matemáticamente el desplazamiento a una magnitud escalar máxima de `1` por turno.

- **Control de Flujo Discreto (`continue`):** Abandona el uso de la instrucción `break` para la validación perimetral. Al no existir un rayo que detener, si el cálculo de la casilla destino (`nextPos`) excede los límites físicos de la matriz (`!isValidPosition`), el motor ejecuta un `continue` para abortar esa evaluación específica y saltar inmediatamente al cálculo del siguiente vector direccional.

- **Resolución Categórica y Binaria:** La validación de la casilla objetivo es absoluta en una sola pasada. Si está vacía, se registra el movimiento pacífico; si está ocupada, se

evalúa su color para registrar una captura condicional. Al no haber iteración continua, no es necesario programar interrupciones por colisión.

4.6. Implementación de la Clase `Knight` (Caballo)

La clase `Knight` completa el grupo de las piezas de movimiento discreto o paso fijo (Step Pieces). Es la única pieza del motor de ajedrez con una traslación asimétrica y no lineal, y la única que posee la capacidad de "saltar" masa física.

Para replicar esta mecánica compleja con la máxima eficiencia computacional, la clase aplica los siguientes principios de diseño:

- **Vectores de Traslación Asimétrica (Movimiento en "L"):** Define una matriz de 8 vectores fijos (`directions[8][2]`) combinando magnitudes dispares de `2` y `1`. Esta permutación matemática perfecta garantiza que el destino siempre represente un cambio de dos casillas en un eje y una casilla en el eje perpendicular, generando la icónica trayectoria en "L".

- **Salto Físico Implícito (Ausencia de Raycasting):** A diferencia de las piezas deslizantes, el algoritmo omite por completo la evaluación de las casillas intermedias. Al calcular directamente la coordenada final sin un bucle de progresión, el motor otorga al Caballo la capacidad intrínseca de ignorar cualquier bloqueo perimetral aliado o enemigo.

- **Control de Flujo Discreto (`continue`):** Comparte la arquitectura de validación del Rey. Si el salto proyectado aterriza fuera de la matriz del tablero (`!isValidPosition`), la instrucción `continue` aborta inmediatamente el cálculo de ese vector y procede al siguiente, previniendo excepciones de memoria.

- **Resolución Categórica:** Al igual que el Rey, el análisis de la casilla de aterrizaje es binario y definitivo: si está vacía, se registra un salto pacífico; si está ocupada por una pieza de color opuesto, se registra una captura válida.

5. Implementación de la Clase `GameState`

La clase `GameState` actúa como el motor transaccional de la partida. Su responsabilidad es coordinar la disposición física de las piezas (`Board`) con las reglas abstractas e históricas (posicion de los reyes, turnos, enroques y captura al paso).

Para garantizar transacciones seguras y un rendimiento óptimo para la IA (Minimax), aplica los siguientes principios de C++ Moderno:

- **Transacciones Atómicas (`std::stack`):** Empaqueta el estado temporal en una estructura `StateInfo` antes de cada movimiento. Esto permite deshacer cualquier jugada (`undoState`) recuperando el contexto histórico en tiempo constante $O(1)$, protegiendo la integridad de los datos.

- **Observación sin Propiedad (`dynamic_cast`):** Utiliza punteros crudos (`Piece*`) para observar las piezas sin alterar la gestión de memoria del tablero. Luego, emplea `dynamic_cast` para identificar polimórficamente su tipo (ej. `<King*>`) de forma segura.

- **Abstracción de Estados Efímeros:** La captura al paso dura un solo turno. Para modelar su ausencia matemáticamente, inicializa `enPassantTarget` con la coordenada "fantasma"`Position(-1, -1)`, creando un validador implícito ultrarrápido.

- **Matemática Unificada (`std::abs`):** Calcula el salto doble del Peón usando el valor absoluto de la diferencia de filas. Esto unifica la ecuación geométrica y elimina la redundancia de programar lógicas separadas para blancas y negras.

- **Inmutabilidad Controlada (`const`):** Blinda la clase marcando sus métodos de consulta (como `canCastle` o `getCurrentTurn`) estrictamente como `const`, previniendo modificaciones accidentales por parte de sistemas externos.

6. Implementación de la Clase `MoveGenerator`

La clase `MoveGenerator` es el motor de exploración táctica del sistema. Su responsabilidad es unificar la lógica física de las piezas (`Piece`) con las reglas abstractas del juego (`GameState`) para generar una lista maestra de movimientos "pseudo-legales".

Para maximizar el rendimiento de la IA (algoritmo Minimax), la implementación aplica rigurosamente principios de C++ Moderno:

- **Pre-asignación de Memoria (`reserve`):** Reserva explícitamente 45 espacios en el `std::vector` de movimientos. Esto evita los costosos realojos dinámicos de memoria en tiempo de ejecución.
- **Fusión Optimizada (`insert`):** Utiliza iteradores (`begin()`, `end()`) para copiar bloques de memoria de forma directa al agrupar los movimientos, descartando los lentos bucles manuales.
- **Inmutabilidad (`const GameState&`):** Recibe el estado por referencia constante, operando en un entorno estricto de solo lectura que evita copias pesadas en RAM y protege la integridad del tablero.
- **Método `generateLegalMoves`** El método `generateLegalMoves` de `MoveGenerator` filtra los movimientos pseudo-legales simulando cada jugada temporalmente con `updateState`. Luego, utiliza `RuleValidator` para comprobar que el Rey aliado no quede en *jaque*. Finalmente, revierte el tablero con `undoState` y retorna únicamente los movimientos seguros.

6.1. Reglas Especiales: Enroques (`generateCastlingMoves`)

Dado que las piezas ignoran el estado global, el generador calcula los enroques mediante una **Validación Espacial Directa**. Consulta las banderas del `GameState` y verifica, en tiempo $O(1)$, que las coordenadas fijas (hardcodeadas) entre el Rey y la Torre estén vacías (`nullptr`), sin recurrir a iteraciones complejas.

6.2. Reglas Especiales: Captura al Paso (`generateEnPassantMoves`)

El algoritmo evalúa matemáticamente la ventana temporal de un solo turno para esta regla:

- **Filtro Rápido:** Descarta instantáneamente el cálculo si la coordenada objetivo `epTarget` es la "fantasma"(fila -1).
- **Escaneo Lateral Absoluto:** Deduce la fila obligatoria de ataque (`pawnRow`) y, utilizando límites perimetrales lógicos, busca piezas aliadas adyacentes a la columna vulnerada.

- **Seguridad de Tipos:** Emplea `dynamic_cast` para confirmar polimórficamente que cualquier pieza aliada encontrada sea estrictamente un `Pawn` antes de registrar el movimiento.

7. Implementación de la Clase `RuleValidator`

El `RuleValidator` actúa como el ´Inspector Matemático´ del motor. Siguiendo el Principio de Responsabilidad Única (SRP), su única función es analizar un `GameState` estático y responder a una consulta booleana: *¿Está el Rey bajo ataque?* La gestión de listas de movimientos se delega por completo al `MoveGenerator`. Esta clase sera `static` para mejorar la optimizacion al no tener instanciación de esta cada turno.

7.1. Algoritmo de Escaneo Inverso (Reverse Raycasting)

En lugar de la aproximación ingenua de calcular todos los movimientos enemigos (lo que exigiría una costosa asignación de memoria dinámica), el algoritmo utiliza el **Escaneo Inverso**. Toma la coordenada del Rey como epicentro y proyecta vectores hacia afuera para detectar colisiones, operando sin instanciar ningún vector en la memoria RAM.

7.2. Mecanismos de Detección (Radares)

La implementación matemática se divide en dos enfoques geométricos:

- **Radar Expansivo (Raycasting):** Diseñado para Torres, Alfiles y Reinas. Proyecta un vector direccional y lo expande iterativamente por la matriz. Si colisiona con un atacante, retorna `true`. Si choca con cualquier otra pieza, esta actúa como escudo y se aplica un `break` para detener el rayo.
- **Radar Puntual:** Diseñado para Caballos, Peones y el Rey enemigo. Al no ser piezas deslizantes, omite la expansión y verifica mediante transformaciones matriciales directas (ej. saltos en "L") si la casilla destino contiene una amenaza, resolviéndose en tiempo $O(1)$.

7.3. Salida Temprana (Early Exit)

El algoritmo está optimizado mediante evaluación perezosa. Al primer hallazgo de un jaque, la función retorna `true` instantáneamente, abortando el escaneo del resto del tablero y ahorrando ciclos de CPU críticos durante la ejecución del algoritmo Minimax.

8. Implementación de la Clase `Player`

Actúa como la clase abstracta base que establece el contrato arquitectónico para cualquier entidad participante en el sistema. Al carecer de implementación en su método de toma de decisiones, impone estrictamente el Principio de Inversión de Dependencias (DIP) y aprovecha el polimorfismo dinámico de C++. Esto permite que el motor central (`Game`) interactúe de forma agnóstica tanto con usuarios físicos a través de la consola, como con agentes heurísticos de Inteligencia Artificial.

- **Estado Fundamental Inmutable:** La clase gestiona la identidad del jugador (`Color`), la cual se asigna durante la construcción y permanece inalterable durante el ciclo de vida de la partida, siendo accesible únicamente mediante consultas de solo lectura (`const`) para proteger la encapsulación.
- **Delegación de Decisiones (`getMove`):** Mediante un método puramente virtual, fuerza a las clases derivadas a definir su propio motor de decisión o mecanismo de entrada. Esta abstracción desplaza la responsabilidad computacional de elegir una jugada válida fuera del controlador principal.

8.1. Implementación de laClase `HumanPlayer`

Es una implementación concreta de la clase `Player` diseñada específicamente para la interacción humana en consola. Actúa como un puente de validación seguro entre la entrada estándar del sistema (`std::cin`) y la lógica interna del motor de ajedrez, garantizando que el juego solo procese intenciones que correspondan estrictamente a movimientos física y lógicamente válidos.

- **Análisis Léxico Ligero (`parsePosition`):** Emplea un analizador que transforma la notación algebraica de ajedrez (ej. ."e2") en índices matriciales $(0 - 7)$. Ejecuta esta conversión en tiempo $O(1)$ utilizando aritmética directa y offsets de la tabla ASCII, invirtiendo matemáticamente la escala de las filas para acoplarse a la matriz interna del tablero.
- **Bucle de Validación Estructural:** Sobrescribe la toma de decisiones (`getMove`) implementando un ciclo interactivo robusto. Este mecanismo limpia los flujos de error del buffer de consola ante entradas malformadas y cruza iterativamente el input del usuario contra la lista pre-calculada de movimientos legales, asegurando la integridad de la jugada antes de retornarla.

9. Implementación de la Clase `Game`

Representa la **fachada principal** de la lógica del ajedrez (Capa de Dominio) y opera como una máquina de estados. Diseñada bajo el Principio de Responsabilidad Única (SRP), delega el manejo de la interfaz a los verdaderos controladores visuales (como `ChessController`). Su rol fundamental es servir de frontera arquitectónica, garantizando que el entorno gráfico consuma las reglas del ajedrez sin acoplarse a su compleja gestión de memoria subyacente.

- **Encapsulación y Despacho Dinámico:** Mantiene localmente el `GameState` y el `MoveGenerator` para calcular el árbol de jugadas legales, delegando la elección mediante polimorfismo inyectado (`Player*`) sin importar si el turno pertenece a un humano o a un algoritmo.
- **Desacoplamiento Estructural (DTOs):** Puesto que la clase `Move` es delicada y gestiona punteros dinámicos (`unique_ptr`), `Game` actúa como filtro. Exporta a los controladores únicamente coordenadas primitivas (`std::vector<Position>`), previniendo fugas de memoria.
- **Procesamiento Transaccional (`processMove`):** Adapta el motor al flujo asíncrono de la GUI. Recibe coordenadas puras, las valida contra el árbol de jugadas, procesa operaciones complejas (ej. coronaciones) y auto-evalúa el fin de partida (Jaque Mate o Ahogado).
- **Consultas Inmutables (*Queries*):** Provee a los controladores una vista constante del tablero (`getBoard`) y métodos reactivos (`isInCheck`, `isGameOver`) para que la interfaz reaccione a eventos lógicos sin permisos de mutación.
- **Compatibilidad Iterativa (`playTurn`):** Preserva un bucle lógico secuencial puro, permitiendo la ejecución de partidas automatizadas o pruebas de estrés en consola mediante una única llamada cíclica.